UC San Diego

Self-Evolving Game Agent with Tool-Integration

CSE 291A - AI Agents

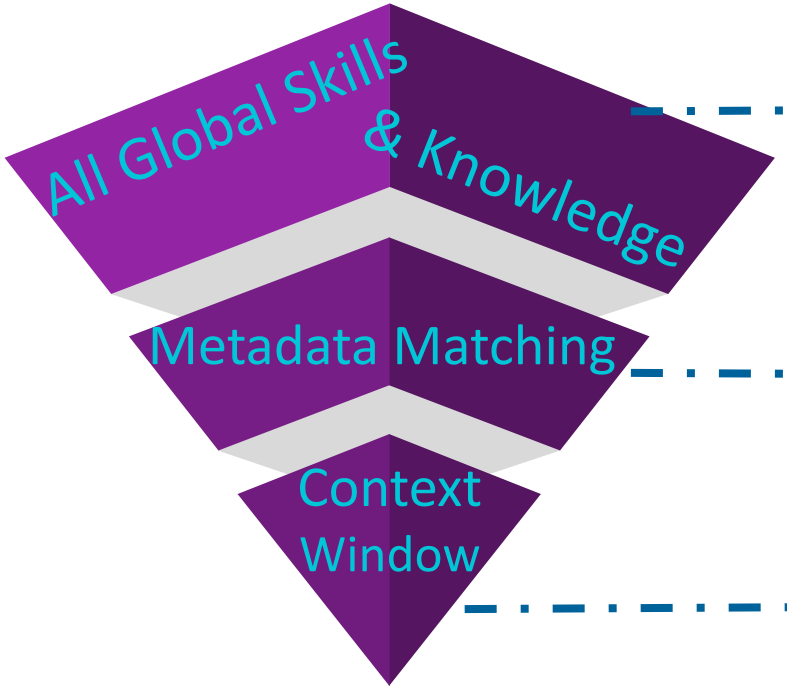
Introduction - Project Overview

- Designed as a **generic solution** for any turn-based game.
- A self-evolving hybrid agent : Integrates **Reinforcement Learning (RL)** for core decision-making with a **LLM subsystem** for environmental reasoning.
- Key Capability:
  - **Tool Creation**: The LLM analyzes the game to generate auxiliary tools that the RL agent uses for new actions
  - **Evolving Tool Case:** Rather than fixed rules, the agent constructs a usable interface incrementally with **minimal prior assumptions**

Background

Background

Context vs. Capability: Progressive Disclosure



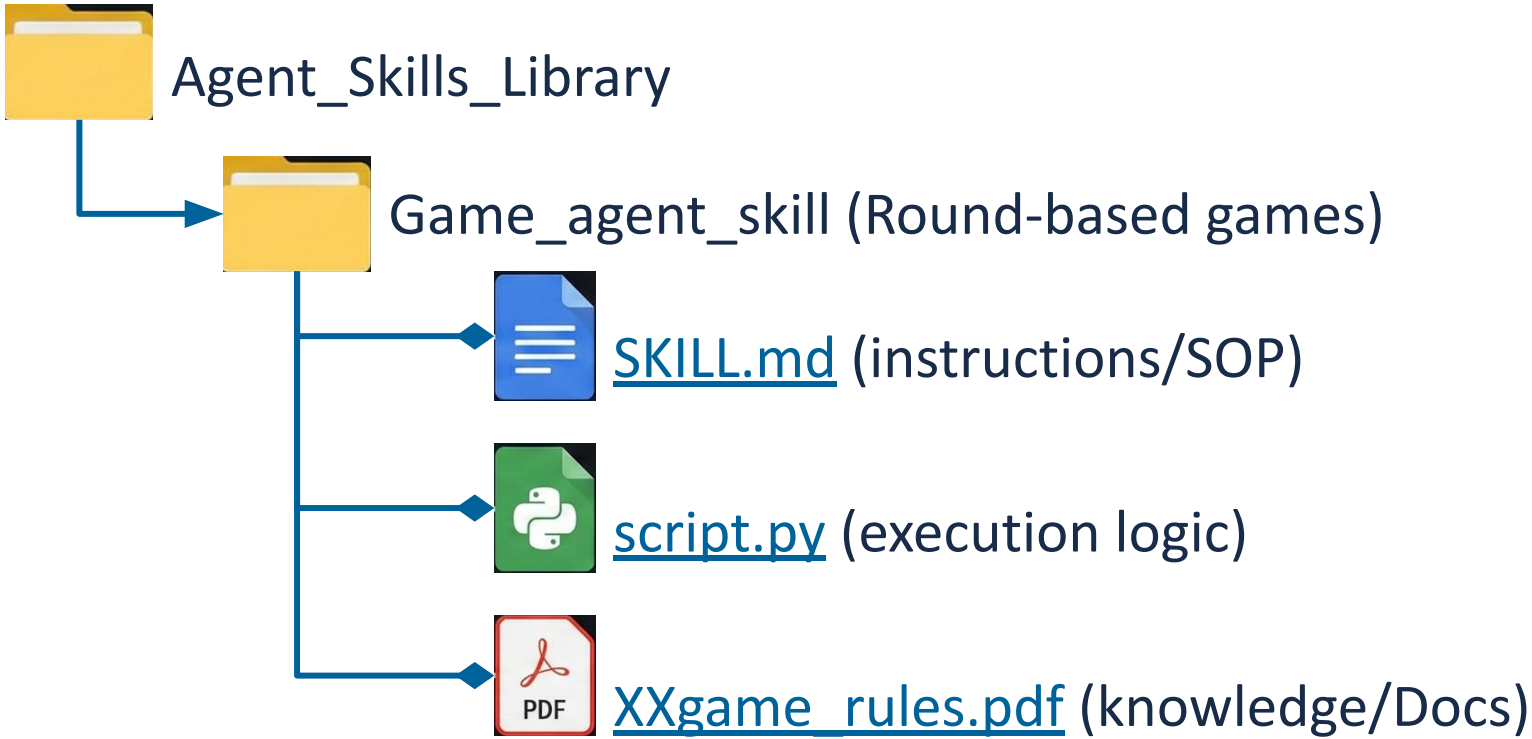
Big Model Capability:
Physics, Coding, History, Medical, etc.
(Big Data, Scaling Laws …)

"Meta Learning":
Expertise, Discipline description ...

Detailed demands:
Loaded for current task.

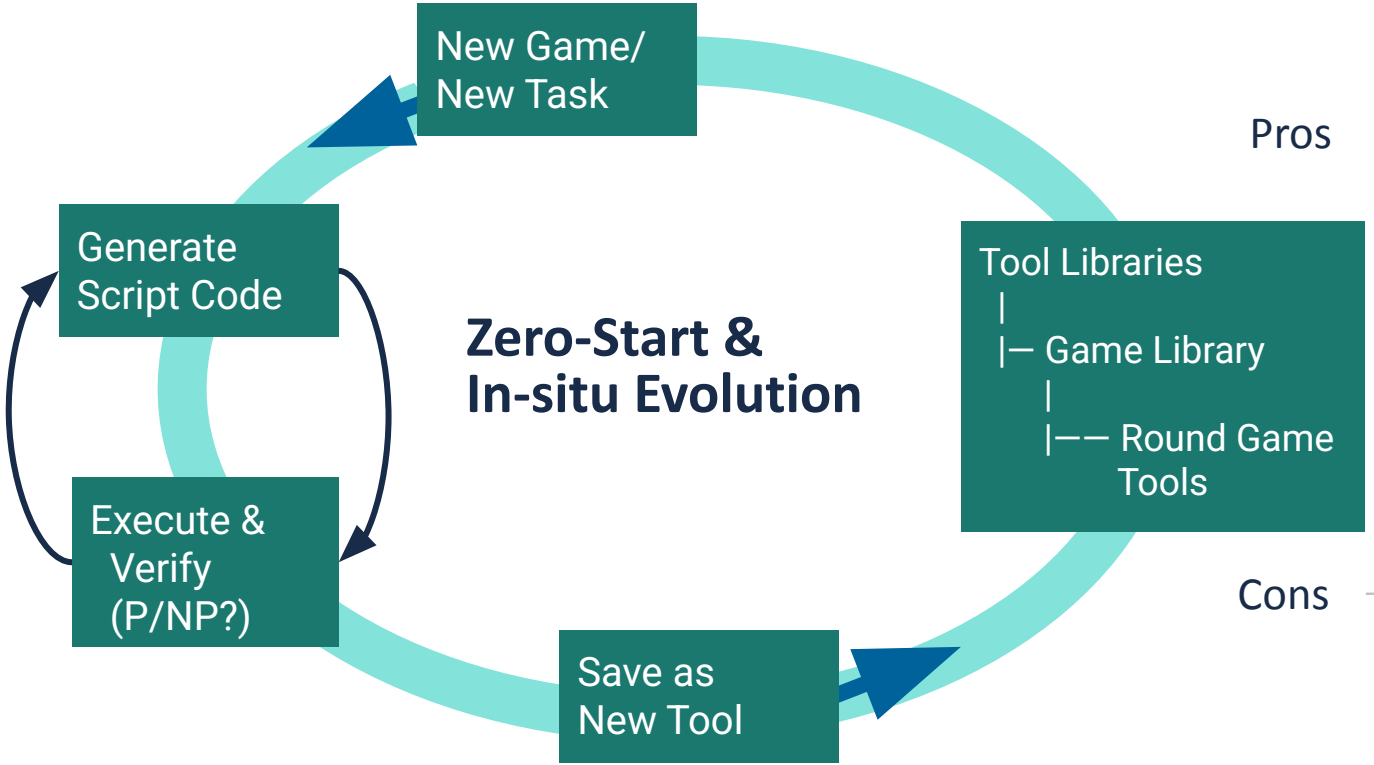
Background

Agent Skills: The Human-Curated Standard



Background

Tool Agent: The Self-Evolving Machine



Zero-Start &
In-situ Evolution

Pros
- Zero-Start Scalability
- Self-Healing
- Infinite Scope

Cons
- Safety Hazard
- Silent Failure
- Depends on binary feedback

Motivation

Motivation 1:Why Tool-Using LLM Agents?

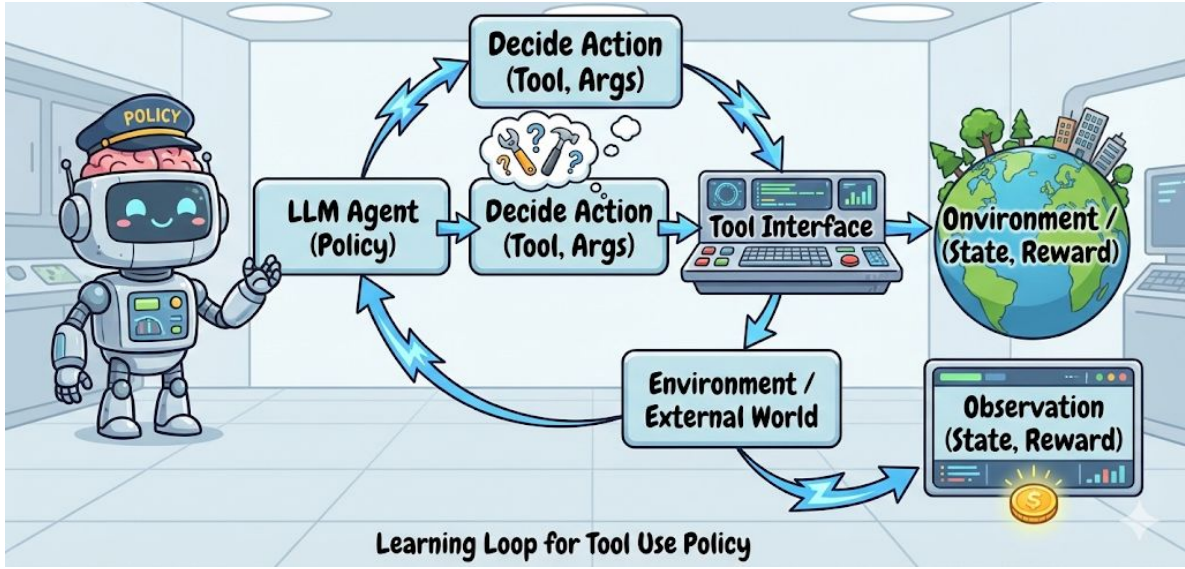
UC San Diego

Current LLM agents use tools via:

- Prompt-based tool invocation
- Hand-crafted rules or heuristics

These approaches are:

- ❌ Expensive (many tool calls)
- ❌ Hard to generalize
- ❌ Environment-specific



Key insight: Tool usage is a ***sequential decision problem*** with long-term rewards.

Motivation: Prompting is insufficient. Tool-use should be learned as a policy.

Motivation 2:Why Games + Yunjue?

UC San Diego

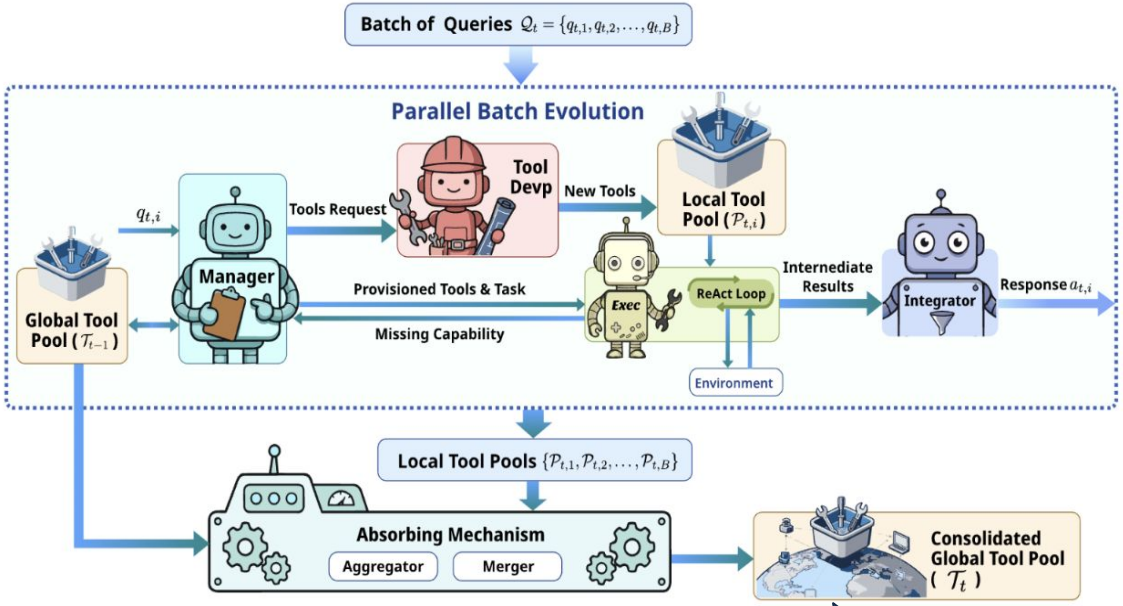
Why Turn-Based Games?

- Clear state–action–reward structure
- Low cost, fully observable
- Rich tool needs: planning, memory, inventory

Turn-based games provide a **clean RL testbed** for studying tool-use.

Why Yunjue?

- Provides a general LLM agent interface
- Supports state analysis and tool integration



❗ **But**: Yunjue does not define ***how agents should learn to use tools***.

Related work

- **Yunjue Agent**
- Article: https://arxiv.org/abs/2601.18226
- Github: https://github.com/YunjueTech/Yunjue-Agent
- An AI agent that can make plan to build skills via python script to solve complex problems, instead of relying on human guidance to plan or build applicable tools
- **Skills**
- https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf?hsLang=en
- Launched by Anthropic on Oct 2025, a skill is a set of instructions that teaches LLM agent to perform a specific task

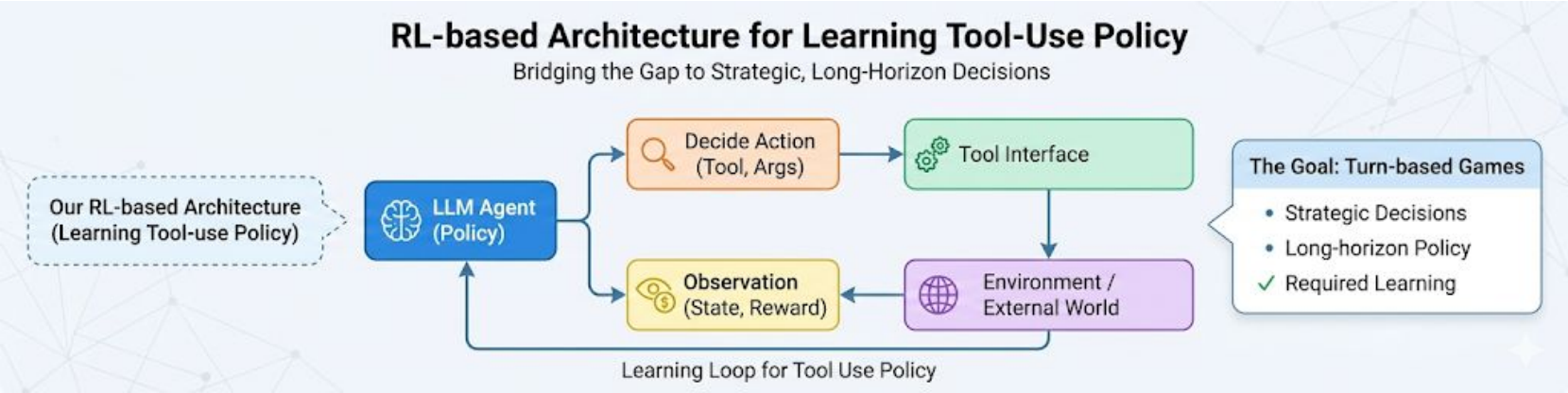
The Research Gap & Our Goal

UC San Diego

The Gap

Feature	Yunjue	Turn-based Games
Tool-use	Abstract API	Strategic Decisions
Decision Type	Manual	Long-horizon Policy
Learning	❌ Not learned	✅ Required

Gap: No mechanism for learning ***when and how*** to use tools in multi-step environments.

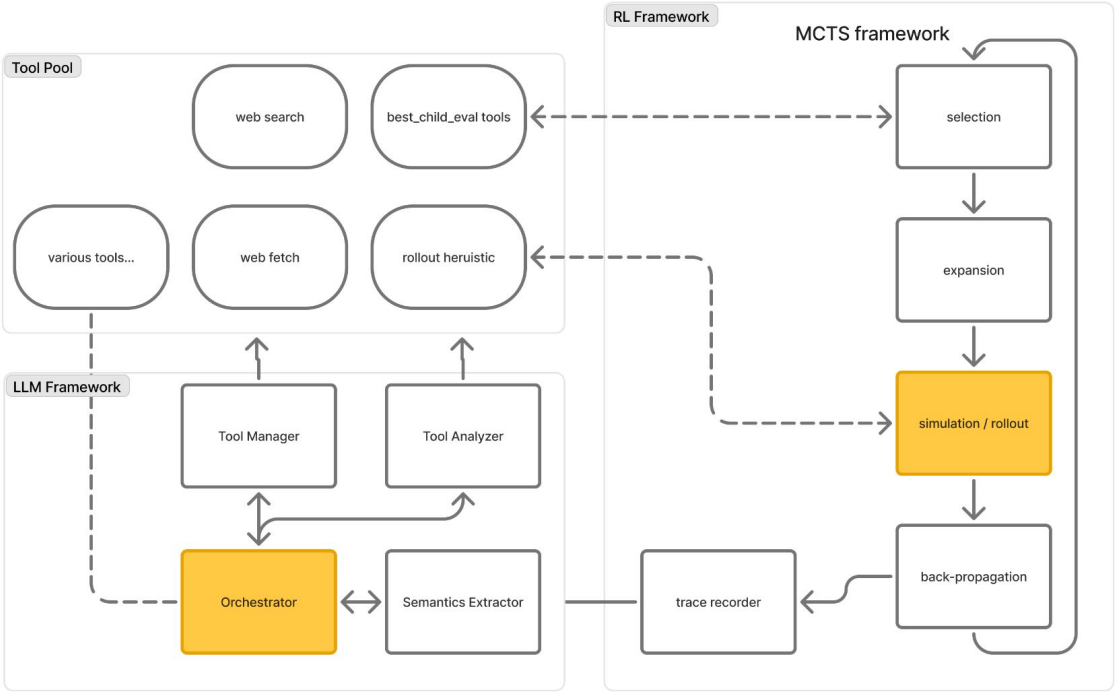


Motivation & Goal: Design a RL-based architecture that enables LLM agents to **analyze, create, and learn to use tools** within Yunjue-based turn-based games.

Impact: Significant exploration of the “skills” concept by attempting to solve large-scale, generalizable problem domain with AI agent

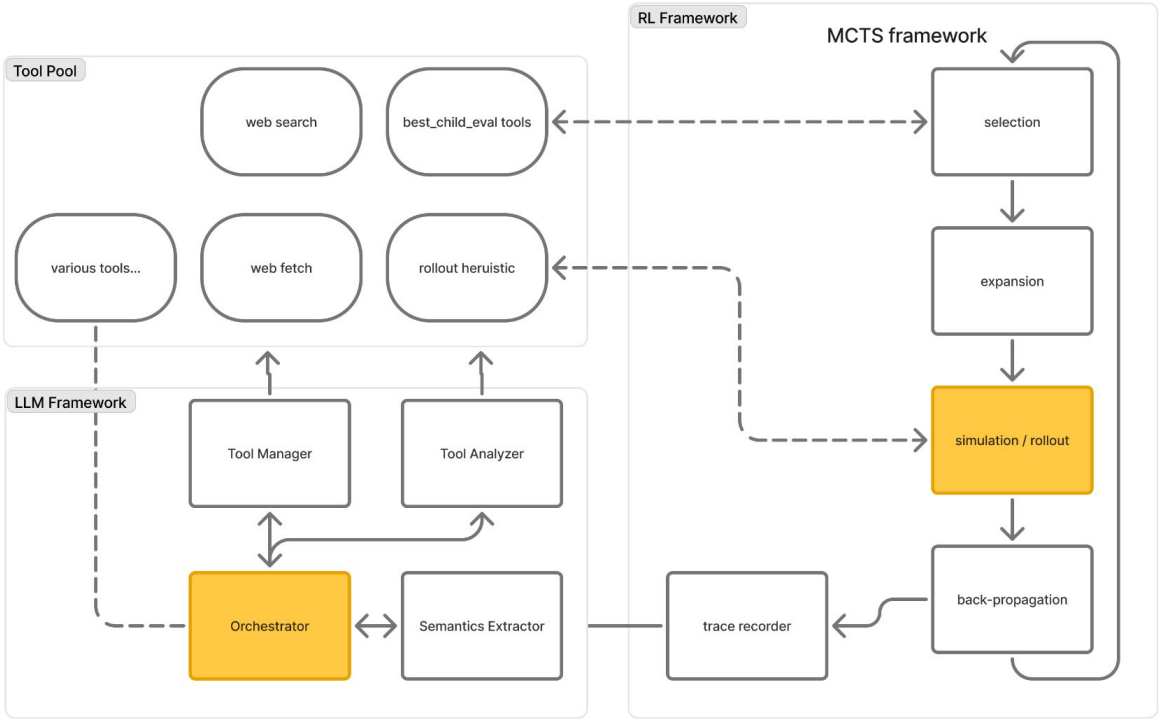
Project Part1: Overall Architecture

Core Designs



- Monte-Carlo Tree Search based RL framework
- Tool use are embedded into RL loop
- Game trace will be recorded and used to optimize tool pool by LLM

CSE291 Architecture



Project Part2: Cross-Game Plan

Cross-game improvement plan

- We plan to continuously improve our agent's tool-kit on a series of games with **increasing complexity.**
- Start with simple games (e.g. connect-4, 2048 etc. from OpenSpiel) to sanity check that tool-use does improve RL performance.
- Migrate the tools onto more complex games while solidifying a toolbox with the tools/heuristics that **transfer** successfully between games.
- Evaluate our agent on games that are costly to perform RL like Quoridor or Chess to show that an effective list of tools can **improve RL performance at low training iterations**.
- With sufficient training iterations, we don't expect to outperform a RL framework like AlphaZero because heuristics are not as flexible. But we believe that our proposed RL agent with LLM synthesized tools/heuristics can perform well on lower training iterations.

MCTS Experiment - Quoridor

- Two players take turn, can either place bar or move in four directions.
- Bad performance with naive MCTS, and need manual heuristic fine-tuning
- Number of states similar to Chess, but most actions have no use

Future Evaluation

We wish to demonstrate:

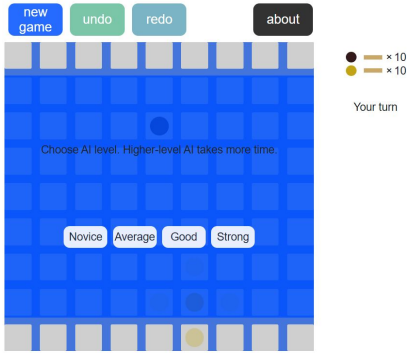
- Superior performance in target game (Ex. Quoridor)
    - Fair measurement based on established, less formal metrics
- Adaptivity to a family of games
    - Eventually, integrated tools should generalize
- Growing performance overtime
    - Better toolbox leads to stronger performance

Case Study Example: Quoridor

- **Initial Research:**
  - Mostly thesis paper, many projects
  - Popular metric: game win rate against other heuristics/agents
- **Our current idea:**
  - Simulate games against existing heuristics or agents, report win rate

p1 \ p2	Random	Semi-Random	A*	Minimax	MCTS
Random		31.3	6.6	5.3	0
Semi-Random	73.5		11.5	10.2	0
A*	92.2	90.2		0	0
Minimax	92.4	88.9	8.2		0
MCTS	90	74.3	100	74	

An Undergraduate Thesis: Artificial Intelligence for Quoridor Game



Level	Rollouts per Move
Novice	2,500
Average	7,500
Good	20,000
Strong	60,000

Online Agent: Quoridor AI with MCTS

Evaluate a Family of Games

- **How to Evaluate Adaptivity?**
  - Start with training model with Quoridor
  - With more tools, train on other games of similar mechanics e.g. Ricochet Robots, Tek (2-player, grid-map, turn-based strategy)
  - Observe usages of established tools and quick improvement of performance
  - Expectation: existing tools help agent to perform better in new games

Table 1: **Comparison between A* and our algorithm.** As the instances get harder the gap between the proportion of solved of A* and our algorithm increases.

Grid Size	#Helpers	#Instances	A*	Our
32 × 32	3	10000	90,8 %	**94,7 %**
96 × 96	3	1000	54 %	**87,8 %**
96 × 96	5	1000	21,2 %	**86 %**

Example: Ricochet Robots Evaluation

1 https://www.scitepress.org/Papers/2023/116999/116999.pdf?utm_source=chatgpt.com

Evaluate Growth Overtime

- **How to Evaluate Impact of New Tools on Performance?**
  - We can evaluate on the same game before and after training all games
- **Current Challenges:**
  - Particular games lack authoritative, high-potential evaluation methods
  - Lacks quantitative evaluation for other advantages
  - Simulations can be expensive
- **Our Next Step:**
  - Continue searching for existing work
  - Explore wider evaluation options like cost/quality tradeoff

Timeline

- **We anticipate the major blockers to be the technical challenges in developing from the new Yunjue agent, establishing outstanding RL architecture, and achieving the generalization and integration of skills.**
- **Week 6: Codebase setup, establish baseline**
  - We expect to settle a workable implementation plan, familiarize major tools like Yunjue Agents, and establish a minimum solution for simple game
- **Week 7: Workable solutions for above major parts**
  - We will complete the implementation of a suitable RL architecture, necessary functionalities based on Yunjue, and prompts or code for optimized tools
- **Week 8: Integration of subparts, optimization and evaluations**
  - We will present an agent that integrates above parts to solve a domain of simpler games through development, optimization, and integration of tools in an efficient way
- **Week 9: Final report and minor fixes**
  - After also complete evaluation in week 8, we will draft the final reports and fill missing details

Cost & Token Usage Prediction: hybrid models

UC San Diego

Phase 1: Implementation & Tuning

Test Games: 50 rounds of testing for a single game

Tokens per Game: ~600k tokens (High estimate, unoptimized).

Cost Calculation (Per 50 Games):

- **Local (Qwen3-7B/14B deployed locally): $0.00**.
- **DeepSeek API (DeepSeek-V3):**
  - *Input:* $0.28 / 1M tokens.
  - *Output:* $0.42 / 1M tokens.
  - *Total Tokens:* 30M tokens (50 games * 600k).
  - *Estimated Cost:* **$9~$12**

Depending on the actual development, we'll spend ~$40 for implementation

Phase 2: Benchmarking & Adaptability Showcasing

Test Games: 3~5 round based games with prior RL cycles

Total RL cycles + Showcasing cycles: ~100 rounds

Tokens per Game: ~600k tokens

Total Volume: 60 Million Tokens.

Cost Calculation (Gemini 3 Flash):

- *Input ($0.5/1M):* 48M * $0.5 = $24
- *Output ($3/1M):* 12M * $3 = $36

Depending on the actual testing, phase 2 total would be $60 ~$80

Cost Prediction: Yunjue case study

UC San Diego

- We will use Yunjue Agent and its embedded GPT-5 model for substantial part of our project, so we will use Yunjue alone to present another sample estimate of our quantitative token expenditure, if we decide to base on Yunjue mostly.
- In previous test case, a prompt involving 4 skills can cost 50k tokens, which is around $0.25 in Openai API platform. Our actual work with simple game should be no less.
- Suppose we be efficient, and allocate 3M tokens ($15) for component development
  - 15 prompts in game prompt development
  - 25 prompts in creating dynamic prompt chain for RL
  - 20 prompts for testing generalizable tools, 10 prompts for testing reward functions
- And we spend 5M ($25) tokens for agent constructions, and 10M ($50) tokens evaluation
  - 100 prompts for integrating components for self-evolving game agent
  - 200 prompts for evaluating agent performance (20 prompts, test set of 10 games)
- So the token cost can be **~18M tokens ($90)** for **GPT-5**. Note that the estimate can differ due to the inaccuracy of initial testing (overestimate token cost) and increased complexity of future development

UC San Diego

THANK YOU!